

Trade Bot Advisor v4.0

Multi-Agent + Dynamic Optimizer + Advanced Reasoning

Product Documentation

Date: 2026-04-30

<https://huggingface.co/spaces/SutskeverFanBoy/trade-bot-advisor>

DISCLAIMER: Educational and research purposes only. Not investment advice.

Table of Contents

1. Executive Summary
2. System Architecture (7-Layer)
3. Core Modules (11 files)
 - 3.1 Signal Engine
 - 3.2 Multi-Agent System (NEW v4.0)
 - 3.3 Dynamic Strategy Optimizer (NEW v4.0)
 - 3.4 Advanced Reasoning (NEW v4.0)
 - 3.5 Backtesting Engine
 - 3.6 Ground Truth Evaluator
 - 3.7 Portfolio Simulator
 - 3.8 Data Ingestion
 - 3.9 Chart Builder
 - 3.10 LLM Interpreter
 - 3.11 MCP Server (NEW v4.0)
4. User Interface (12 Tabs)
5. Technical Indicators
6. Signal Rules & Voting
7. Risk Management
8. Data Sources & Fallback
9. Supported Assets
10. v4.0 New Features Deep Dive
11. MCP Integration — Model Context Protocol
12. Performance Metrics
13. Limitations
14. Future Roadmap
15. Tech Stack

1. Executive Summary

Trade Bot Advisor v4.0 is a multi-agent algorithmic trading signal system with self-optimizing strategy weights and advanced contextual reasoning. It represents a major evolution from single-engine signal generation to a collaborative agent architecture where specialized agents analyze different aspects of the market independently, and an orchestrator synthesizes their findings into a final decision.

Key v4.0 additions over v3.5:

- Multi-Agent System: 4 specialized agents (Analyst, Sentiment, Risk, Orchestrator) with configurable weights and risk veto capability
- Dynamic Strategy Optimizer: Uses ground truth data to learn which indicators perform best, then auto-adjusts voting weights
- Advanced Reasoning: 5 contextual analyses (market regime, liquidity traps, divergences, macro context, exhaustion signals) that adjust confidence beyond what raw indicators show

The system now has 12 interactive tabs, 18 files totaling ~250KB, and processes data from yfinance, Binance, and a built-in sample generator that guarantees the demo always works. All agent capabilities are also exposed via MCP (Model Context Protocol), allowing any external AI client (Claude Desktop, Cursor, smolagents) to call the tools programmatically.

2. System Architecture

7-layer architecture with clear separation:

- Layer 1 - Data (data_ingest.py): yfinance + Binance REST + sample fallback. Never returns None.
- Layer 2 - Signal Engine (signal_engine.py): 5-indicator voting (RSI, SMA, MACD, Bollinger, Volume) + volatility filter + confidence scoring + ATR-based position sizing.
- Layer 3 - Multi-Agent (agents.py): 4 independent agents (Analyst, Sentiment, Risk) + Orchestrator that merges with weighted scoring. Risk Agent has veto power.
- Layer 4 - Optimization (optimizer.py + evaluator.py): Ground truth evaluation per indicator. Measures historical accuracy. Computes adaptive weights.
- Layer 5 - Reasoning (reasoning.py): Market regime detection, liquidity trap detection, RSI divergence, macro context, exhaustion signals. Adjusts confidence.
- Layer 6 - Presentation (app.py, charts.py): 12-tab Gradio UI, Plotly charts, JSON output.
- Layer 7 - MCP (mcp_server.py + Gradio native): All tools exposed via Model Context Protocol. Dual integration: Gradio auto-MCP (13 tools at /gradio_api/mcp) + standalone FastMCP server (10 curated tools with Pydantic schemas).

File Structure (18 files, ~250KB)

File	Size	Purpose
app.py	51KB	Main Gradio app, 12 tabs, MCP server enabled
mcp_server.py	15KB	NEW: Standalone MCP server (10 tools)
core/signal_engine.py	23KB	Deterministic 5-indicator signal generator
core/agents.py	17KB	Multi-agent system (4 agents + orchestrator)
core/optimizer.py	8KB	Dynamic strategy weight optimizer
core/reasoning.py	16KB	Advanced contextual reasoning engine
core/backtester.py	9KB	Walk-forward backtesting
core/evaluator.py	8KB	Ground truth evaluator
core/portfolio.py	11KB	Stateful portfolio simulator
core/data_ingest.py	6KB	Data fetch: yfinance + Binance + fallback
core/charts.py	12KB	Plotly chart builders (8 chart types)
core/llm_interpreter.py	5KB	Optional LLM explanation layer

3. Core Modules

3.1 Signal Engine (signal_engine.py)

The deterministic brain. 5-indicator voting: RSI(14), SMA(20/50) crossover, MACD(12,26,9), Bollinger Bands(20,2), Volume ratio. Voting: 3+/5 bullish=BUY, 4+=STRONG_BUY, same for bearish. Extreme volatility filters out BUY signals. Confidence: 0-100 based on agreement, RSI confirmation, volume, volatility. Position sizing: ATR-based SL/TP with risk % per tolerance level.

3.2 Multi-Agent System (agents.py) — NEW v4.0

4 specialized agents, each producing independent AgentReport with signal (BULLISH/BEARISH/NEUTRAL), confidence (0-100), reasoning chain, and data payload.

AnalystAgent

Runs the full signal engine. Extracts technical indicators, votes, bullish/bearish counts. Pure quantitative analysis.

SentimentAgent

Analyzes market mood via: 5-day/20-day price momentum, volume trend (rising/falling/average), volatility expansion/contraction. Produces composite sentiment score. In production: would integrate FinBERT on real news headlines.

RiskAgent

Evaluates: annualized volatility, recent drawdown from peak, ATR gap risk, trend strength (ADX proxy). Produces verdict: APPROVE (safe), CAUTION (reduce size), REJECT (don't trade). Risk score 0-5+.

OrchestratorAgent

Merges all 3 reports using configurable weights (default: Analyst 50%, Sentiment 25%, Risk 25%). Scores: BULLISH=+1, NEUTRAL=0, BEARISH=-1, weighted by confidence. Risk veto: REJECT overrides BUY to HOLD. CAUTION halves conviction. Consensus detection: UNANIMOUS/MAJORITY/SPLIT affects confidence bonus.

3.3 Dynamic Strategy Optimizer (optimizer.py) — NEW v4.0

Self-improving loop: walks through 2 years of historical data, day by day. At each day, measures each indicator's individual accuracy against N-day future returns. Computes per-indicator accuracy (RSI: X%, SMA: Y%, MACD: Z%, Bollinger: W%, Volume: V%). Generates optimized weights proportional to accuracy (minimum 5% weight per indicator). Supports multi-ticker averaging.

Example output: Default weights RSI=20%, SMA=20%, MACD=20%, Bollinger=20%, Volume=20%. Optimized: RSI=15%, SMA=15%, MACD=8%, Bollinger=13%, Volume=49% — because Volume had 70% accuracy while MACD had only 44%.

3.4 Advanced Reasoning (reasoning.py) — NEW v4.0

5 contextual analysis modules that go beyond 'RSI is oversold':

- Market Regime Detection: Classifies as STRONG_UPTREND, STRONG_DOWNTREND, VOLATILE, RANGING, or TRANSITIONAL. Based on up-day percentage, volatility, 20-day price range.
 - Liquidity Trap Detection: Bull trap = price near highs + declining volume. Bear trap = price near lows + declining volume. Warns when breakout may be fake.
 - RSI-Price Divergence: Bearish divergence = price near highs + RSI overbought. Bullish divergence = price near lows + RSI oversold. Signals weakening momentum.
 - Macro Context: Asset-class-aware. Crypto: risk-on/risk-off. BIST: TRY pressure/TCMB. US stocks: goldilocks/correction/uncertain. Commodities: inflation-sensitive.
 - Exhaustion Detection: Extreme RSI + volume spike = buying/selling exhaustion. Potential top/bottom forming.
- Synthesis engine adjusts confidence: regime alignment (+/-10-15%), trap warnings (-10%), divergence (-5%), exhaustion (-8%). Final verdict: STRENGTHENED, CONFIRMED, or WEAKENED.

3.5 Backtesting Engine (backtester.py)

Walk-forward backtest with same signal logic. Configurable RSI thresholds, SMA/vol filters, holding period, evaluation period. Metrics: accuracy, total return, Sharpe, max drawdown, profit factor. Equity curve tracking. Buy & hold comparison.

3.6 Ground Truth Evaluator (evaluator.py)

For every historical signal: records actual price change at 5, 10, 20 day horizons. Per-signal verdict: correct/incorrect. Aggregate: accuracy per horizon, BUY vs SELL accuracy, directional return. Calibration: high-conviction (extreme RSI) vs low-conviction accuracy comparison.

3.7 Portfolio Simulator (portfolio.py)

In-memory stateful portfolio. Opens/closes positions based on signals. Tracks: cash, equity, PnL, win rate, profit factor, max drawdown. Risk controls: max 95% utilization, no duplicates, SL/TP tracking. Supports multi-ticker simulation.

3.8-3.10 Supporting Modules

Data Ingestion (data_ingest.py): 3-tier fetch (Binance for crypto, yfinance for stocks, sample fallback). Never crashes.

Chart Builder (charts.py): 8 Plotly chart types — candlestick+SMA+volume, RSI panel, MACD panel, equity curve, signal map, portfolio equity, PnL distribution, drawdown.

LLM Interpreter (llm_interpreter.py): Optional HF Inference API (Qwen 72B). Template fallback when no token.

Explains signals, never generates them.

3.11 MCP Server (mcp_server.py) — NEW v4.0

Standalone Model Context Protocol server exposing all agent capabilities as callable tools. Built with FastMCP (mcp >= 1.0). Any MCP-compatible client can connect: Claude Desktop, Cursor, smolagents, custom agents.

10 curated tools with rich Pydantic Field schemas:

- analyze_technical: Full RSI/SMA/MACD/Bollinger/Volume analysis with trade parameters
- analyze_sentiment: Market mood from momentum + volume trends
- assess_risk: Volatility, drawdown, ATR risk with APPROVE/CAUTION/REJECT verdict
- multi_agent_decision: 4-agent orchestrated analysis with configurable weights
- run_backtest: Walk-forward strategy backtesting with Sharpe, drawdown, PF
- ground_truth_eval: Historical predictions vs actual outcomes at 5/10/20d
- optimize_strategy: Per-indicator accuracy + adaptive weight computation
- advanced_reasoning: Regime, traps, divergence, macro, exhaustion analysis
- quick_signal: Lightweight signal (just essentials: signal, confidence, RSI)
- compare_tickers: Multi-ticker side-by-side comparison

Supports 3 transports: stdio (Claude Desktop), SSE, streamable-http (recommended). Additionally, Gradio's native mcp_server=True exposes all 13 UI functions as MCP tools automatically at /gradio_api/mcp.

4. User Interface — 12 Tabs

1. Analiz

Single-ticker: candlestick, RSI/MACD, structured JSON, interpretation

2. Backtest

Configurable strategy backtesting with equity curve, signal map, metrics

3. Karsilastirma

2-5 ticker side-by-side comparison table

4. Explainability

Why this trade? Indicator breakdown, vote count, trade details

5. AI Decision

Multi-ticker pipeline: signal + backtest validation + portfolio execution

6. Evaluation

Dashboard: last 50 signals, equity curve, drawdown, signal distribution

7. Portfoy Sim

Stateful portfolio: open/close trades, PnL tracking, equity curve

8. Ground Truth

Model ne dedi? Gercek ne oldu? Per-signal 5/10/20 day outcomes

9. Multi-Agent (v4.0)

4-agent analysis with adjustable weights and reasoning chain

10. Optimizer (v4.0)

Dynamic indicator weight optimization from ground truth

11. Reasoning (v4.0)

Contextual analysis: regime, traps, divergence, macro, exhaustion

12. Rehber

Usage guide, signal rules, supported assets

5. Technical Indicators

RSI(14)

Wilder smoothing. >70 overbought, <30 oversold, >60 bullish, <40 bearish.

SMA(20/50) Crossover

Golden cross: SMA20>SMA50 + price above = STRONG_BULLISH. Death cross: opposite.

MACD(12,26,9)

EMA12-EMA26=MACD line. 9-period signal. Histogram crossovers detected.

Bollinger(20,2)

20-SMA +/-2 std. Price above upper=overbought, below lower=oversold.

Volume Ratio

Current/20-day avg. >1.5x=bullish confirm, <0.5x=weak signal.

ATR(14)

Average True Range for SL/TP distance and position sizing.

Volatility Regime

Annualized from 30-day returns. LOW<15%, MOD<30%, HIGH<50%, EXTREME>50%.

6. Signal Rules

5-indicator majority vote: 3+/5 bullish=BUY, 4+=STRONG_BUY. Same for bearish=SELL. Extreme vol + BUY -> HOLD (safety). Confidence: base 50, +25 for 4+ agreement, +/-10 RSI confirm, +5 volume, -15 extreme vol.

7. Risk Management

Conservative: 1% risk, 1.5x ATR SL, 2x TP. Moderate: 2%, 2x, 3x. Aggressive: 4%, 3x, 4.5x. Max position: 20% portfolio. In multi-agent: Risk Agent can VETO (override BUY to HOLD) or CAUTION (halve conviction).

8. Data Sources

Primary: yfinance (stocks/ETFs/commodities/forex). Secondary: Binance REST (crypto, no API key). Fallback: seeded random walk sample data — guarantees demo always works.

9. Supported Assets

Type	Format	Examples
US Stocks	TICKER	AAPL, NVDA, MSFT, GOOGL, TSLA, AMZN, META
BIST	TICKER.IS	THYAO.IS, GARAN.IS, AKBNK.IS
Crypto	TICKER-USD	BTC-USD, ETH-USD, SOL-USD
Commodities	Futures	GC=F (Gold), SI=F (Silver)
ETFs	TICKER	SPY, QQQ
Forex	XXX=X	USDTRY=X

10. v4.0 New Features Deep Dive

A. Multi-Agent Architecture

Problem: Single signal engine can't distinguish between 'technical says BUY' and 'market conditions are too risky for BUY'. Solution: Split into 3 specialized agents + orchestrator.

Analyst: Pure technical analysis via signal engine. Sentiment: Price momentum + volume trends + volatility momentum. Risk: Volatility regime + drawdown + ATR risk + trend strength.

Orchestrator merge: $\text{weighted_score} = \text{sum}(\text{signal_value} * \text{weight} * \text{confidence})$ per agent. Risk veto: if Risk says REJECT, BUY -> HOLD. CAUTION halves conviction.

UI: Adjustable weight sliders for each agent. Full reasoning chain visible.

B. Dynamic Strategy Optimizer

Problem: Fixed equal weights (20% each) ignore that some indicators work better in current conditions. Solution: Walk through 2 years of history. At each day, test each indicator independently. Compute accuracy per indicator.

RSI accuracy: Out of N days RSI said bullish, how many times did price go up in 10 days? Same for SMA, MACD, Bollinger, Volume.

Weight formula: $\text{weight} = \max(0.05, (\text{accuracy} - 40) / 100)$, then normalize to $\text{sum}=1$. Minimum 5% weight prevents any indicator from being fully ignored.

Multi-ticker: Run on multiple tickers, average the weights across all tickers for a more robust estimate.

C. Advanced Reasoning Engine

Problem: 'RSI is 32, signal is BUY' doesn't answer 'Is this a liquidity trap?' or 'Does the macro environment support this?'. Solution: 5 contextual analysis modules.

1. Market Regime: Is it trending, ranging, or volatile? Ranging markets -> mean-reversion signals better. Trending -> trend-following better.
2. Liquidity Trap: Price near highs + volume declining = potential bull trap. Warns before fake breakouts.
3. RSI Divergence: Price making new highs but RSI overbought = bearish divergence. Weakening momentum.
4. Macro Context: Asset-class-specific. Crypto in risk-off? BIST under TL pressure? US in goldilocks or correction?
5. Exhaustion: Extreme RSI + volume spike = climactic buying/selling. Potential reversal.

Synthesis: Adjusts original confidence. Regime alignment: +/-10-15%. Warnings: -5 to -10% each. Final verdict: STRENGTHENED/CONFIRMED/WEAKENED.

11. MCP Integration — Model Context Protocol

What is MCP?

MCP (Model Context Protocol) is an open standard by Anthropic for connecting AI models to external tools. It allows any MCP-compatible client (Claude Desktop, Cursor, smolagents, Windsurf, Zed, custom agents) to discover and call the Trade Bot's analysis tools via a standardized JSON-RPC protocol over SSE or HTTP.

Dual MCP Architecture

The system provides two MCP integration layers:

Layer 1: Gradio Native MCP (Automatic)

Enabled via `mcp_server=True` in Gradio's `launch()`. Automatically exposes all 13 tab functions as MCP tools. Zero additional code required. Endpoint: `/gradio_api/mcp/sse` (SSE transport).

Tools auto-generated from Gradio UI functions: `run_analysis`, `run_backtest_ui`, `compare_tickers`, `explain_trade`, `run_ai_trade_decision`, `run_evaluation_dashboard`, `run_portfolio_action`, `get_portfolio_status`, `reset_portfolio_ui`, `run_ground_truth_ui`, `run_multi_agent_ui`, `run_optimizer_ui`, `run_reasoning_ui`.

Layer 2: Standalone FastMCP Server (Curated)

`mcp_server.py`: A purpose-built MCP server with 10 tools, each with rich Pydantic Field schemas (descriptions, constraints, defaults). Better tool names, precise parameter documentation, structured return types. Can run independently of Gradio.

10 MCP Tools (Standalone Server)

Tool	Description
<code>analyze_technical</code>	Full OHLCV technical analysis: RSI, SMA, MACD, Bollinger, Volume, t
<code>analyze_sentiment</code>	Market mood via 5d/20d momentum, volume trend, volatility expansion
<code>assess_risk</code>	Risk evaluation: volatility, drawdown, ATR, trend. APPROVE/CAUTION/
<code>multi_agent_decisio</code>	4-agent orchestrated analysis with configurable weights + risk veto
<code>run_backtest</code>	Walk-forward backtesting: accuracy, Sharpe, drawdown, profit factor
<code>ground_truth_eval</code>	Predictions vs actuals at 5/10/20 day horizons. Per-signal verdicts
<code>optimize_strategy</code>	Per-indicator accuracy from ground truth. Adaptive weight optimizat
<code>advanced_reasoning</code>	Contextual: market regime, traps, divergence, macro, exhaustion
<code>quick_signal</code>	Lightweight: just signal, confidence, risk, price, reason, RSI
<code>compare_tickers</code>	2-5 ticker side-by-side comparison (signal, indicators, risk)

Transport Options

stdio: For Claude Desktop integration. Run: `python mcp_server.py stdio`

SSE (Server-Sent Events): GET `/sse` + POST `/messages/`. Run: `python mcp_server.py sse`

streamable-http (recommended): POST `/mcp`. Run: `python mcp_server.py streamable-http`

Client Configuration Examples

Claude Desktop (claude_desktop_config.json)

```
{"mcpServers": {"trade-bot": {"command": "python", "args": ["/path/to/mcp_server.py]}}}
```

Remote HTTP Client

```
{"mcpServers": {"trade-bot": {"url": "https://your-server:8000/mcp", "transport": "streamable-http"}}}
```

Gradio MCP (SSE)

Endpoint: https://SutskeverFanBoy-trade-bot-advisor.hf.space/gradio_api/mcp/sse

Protocol: SSE transport. Connect via any MCP SSE client.

Tool Schema Design

Each standalone tool uses `Annotated[type, Field(description=..., ge=..., le=...)]` for rich JSON Schema generation. Example:

```
@mcp.tool()
def analyze_technical(
    ticker: Annotated[str, Field(description='Ticker symbol')],
    portfolio_value: Annotated[float, Field(description='Portfolio USD', gt=0)] = 100000,
    risk_tolerance: Annotated[str, Field(description='conservative/moderate/aggressive')] = 'moderate',
) -> dict: ...
```

This generates JSON Schema with descriptions, default values, constraints (`gt`, `ge`, `le`) — enabling AI clients to understand parameter semantics and validate inputs before calling.

Why MCP Matters for This Project

- Plug-and-play extensibility: new agents/tools added once, automatically available to all MCP clients
- Standardized interface: no custom API design needed, MCP handles discovery + invocation
- AI-native: LLM clients (Claude, GPT, Qwen) can chain tools naturally in conversation
- Separation of concerns: Gradio UI for humans, MCP for AI agents, same backend
- Future-proof: as MCP ecosystem grows (more clients, registries), tools are instantly accessible

12. Performance Metrics

Backtesting: accuracy, total return, Sharpe ratio, max drawdown, profit factor, avg win/loss.

Portfolio: equity, PnL, win rate, profit factor, max drawdown.

Ground Truth: per-horizon accuracy (5d/10d/20d), BUY vs SELL, directional return, calibration.

Optimizer: per-indicator accuracy, optimized vs default weights, ranking.

Multi-Agent: per-agent signal/confidence/weight/contribution, consensus type, reasoning chain.

13. Limitations

- No real-time streaming (point-in-time analysis)
- In-memory state (resets on Space restart)
- Sample data fallback (synthetic when live unavailable)
- No short selling in portfolio sim
- Daily timeframe only (no intraday)
- No fundamental data (P/E, earnings)
- No real news/sentiment API (simulated via momentum)
- Sentiment agent uses price-based proxy, not FinBERT on headlines
- Optimizer recomputes each time (no persistent learned weights)
- Risk Agent thresholds are fixed (not adaptive)

14. Future Roadmap

High Priority

- Persistent SQLite for portfolio + learned weights
- Real FinBERT sentiment on DuckDuckGo headlines
- Multi-timeframe analysis (daily + 4h + 1h)
- Walk-forward parameter optimization

Medium

- Real macro data integration (TCMB, Fed, yields)
- Options flow as signal modifier
- Telegram/Discord alert notifications
- REST API for programmatic access

Exploratory

- RL for dynamic position sizing
- Ensemble strategies
- Cross-asset correlation signals
- Mobile app

15. Tech Stack

Python 3.13 | Gradio 6.x (ssr_mode=False, mcp_server=True) | Plotly 6.x | yfinance 1.3+ | Binance REST | NumPy | Pandas | Pydantic 2.x | MCP SDK 1.27+ (FastMCP) | HF Inference API (optional, Qwen 72B) | HF Spaces (free tier)

Total: 18 files, ~250KB. Zero config: git clone + pip install + python app.py
MCP: python mcp_server.py streamable-http (standalone) or auto via Gradio